

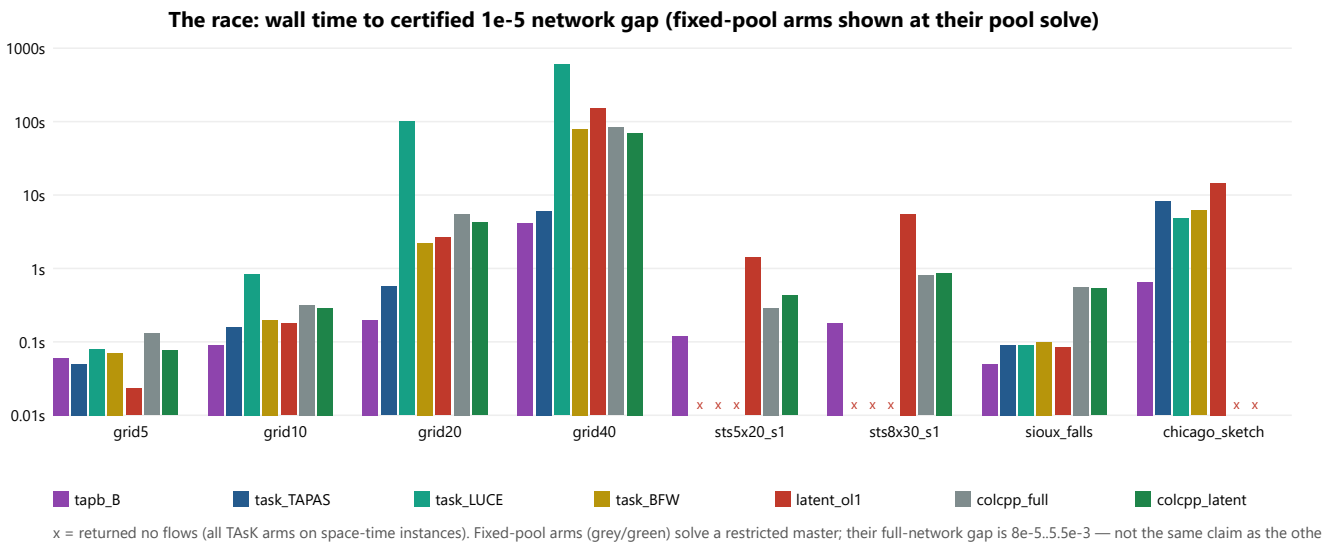
STAP Top-7 Comparison and Latent-Atom Performance Report

Internal — not for release. Generated 2026-08-01 by scripts/build_top7_report.py; all numbers reproduce from the committed harnesses.

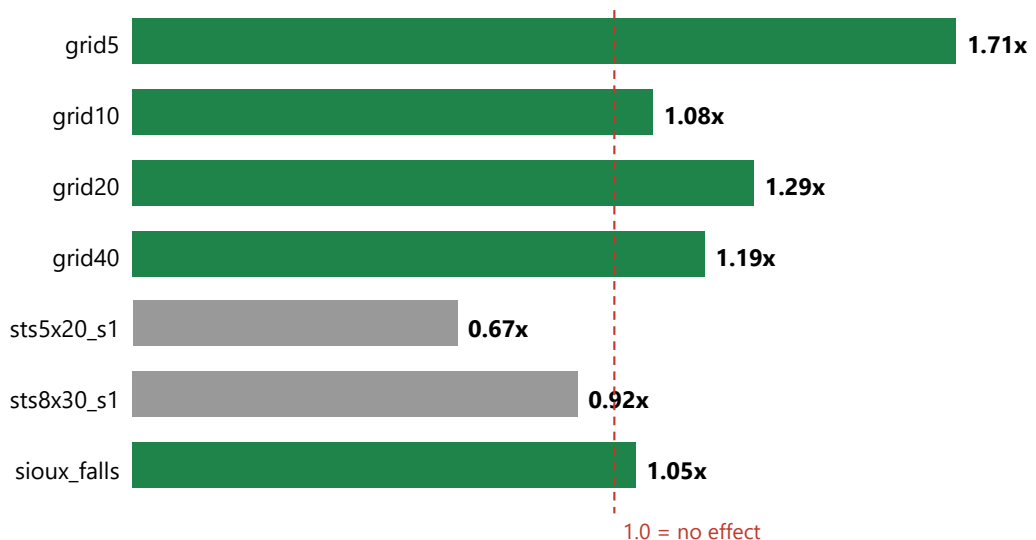
Protocol

Gap target $1e-5$. Seven algorithm arms: **tapb_B** (Dial's Algorithm B, C), **task_TAPAS** / **task_LUCE** / **task_BFW** (TAsK, C++), **latent_o1** (adaptive latent-atom GP, $k=8$ majors, Python kernel, full-network pricing), **colcpp_full** (Jayakrishnan-style GP + elimination, CompressedTAP C++ engine, fixed $K=32$ penalty-KSP pool) and **colcpp_latent** (same engine, explicit majors + one nonnegative atom per OD). Each instance also carries a **decomposition row**: H0 (Bertsekas-style full-simplex projected gradient) / R0 (full GP, elimination) / R1 (latent) with the identity $S_R \times S_{C|R} = S_{RC}$. Every flow-producing run is re-certified by the independent validator (recomputed costs, Beckmann objective, conservation, shortest-path bound). Fixed-pool gaps are restricted-master gaps; the full-network gap is always recomputed separately — the two are never conflated.

The two demonstrations



The critical isolation: same C++ engine, same $K=32$ pool — atom ON vs OFF



Objective error from the atom: $1e-6$ level everywhere — the speedup is free in accuracy terms.

forge/ladder_grid5 — grid 5x5, route-rich, high congestion

arm	time (s)	network gap (recomputed)	pool gap	TSTT	certified
tapb_B	0.06	4.01e-06	—	56,291	True
task_TAPAS	0.05	1.96e-07	—	56,291	True
task_LUCE	0.08	8.17e-06	—	56,291	True
task_BFW	0.07	9.66e-06	—	56,290	True
latent_ol1	0.023	9.63e-06	—	56,290	True
colcpp_full (K=32 pool)	0.13	7.99e-05	9.13e-06	56,292	conservation ok
colcpp_latent (K=32 pool)	0.076	7.82e-05	7.59e-06	56,292	conservation ok

Decomposition (Bertsekas-PG H0 / Jayakrishnan-GP R0 / latent R1): S_R = 0.287193, S_C|R = 3.28013, S_RC = 0.942033; atom objective error 3.89202e-06

forge/ladder_grid10 — grid 10x10, route-rich

arm	time (s)	network gap (recomputed)	pool gap	TSTT	certified
tapb_B	0.09	9.00e-06	—	275,336	True
task_TAPAS	0.16	7.18e-06	—	275,339	True
task_LUCE	0.84	9.46e-06	—	275,337	True
task_BFW	0.2	8.45e-06	—	275,338	True
latent_ol1	0.179	9.60e-06	—	275,336	True
colcpp_full (K=32 pool)	0.316	1.44e-03	9.51e-06	275,514	conservation ok
colcpp_latent (K=32 pool)	0.292	1.44e-03	7.87e-06	275,512	conservation ok

Decomposition (Bertsekas-PG H0 / Jayakrishnan-GP R0 / latent R1): S_R = 1.53729, S_C|R = 3.78046, S_RC = 5.81167; atom objective error 2.30845e-06

forge/ladder_grid20 — grid 20x20, route-rich

arm	time (s)	network gap (recomputed)	pool gap	TSTT	certified
tapb_B	0.2	7.79e-06	—	2,963,617	True
task_TAPAS	0.57	2.71e-06	—	2,963,624	True
task_LUCE	100.14	9.98e-06	—	2,963,622	True
task_BFW	2.21	9.78e-06	—	2,963,645	True
latent_ol1	2.65	9.82e-06	—	2,963,612	True
colcpp_full (K=32 pool)	5.491	2.32e-03	9.96e-06	2,968,587	conservation ok
colcpp_latent (K=32 pool)	4.244	2.31e-03	8.26e-06	2,968,568	conservation ok

Decomposition (Bertsekas-PG H0 / Jayakrishnan-GP R0 / latent R1): S_R = 3.1613, S_C|R = 1.37691, S_RC = 4.35283; atom objective error 2.33335e-06

forge/ladder_grid40 — grid 40x40, route-rich (1,756 nodes)

arm	time (s)	network gap (recomputed)	pool gap	TSTT	certified
tapb_B	4.17	5.26e-06	—	45,908,812	True
task_TAPAS	6.04	6.99e-06	—	45,908,760	True
task_LUCE	601.22	1.67e-04	—	45,910,448	True
task_BFW	79.91	9.50e-06	—	45,908,838	True
latent_ol1	150.274	9.61e-06	—	45,908,763	True
colcpp_full (K=32 pool)	83.012	5.51e-03	9.80e-06	46,050,481	conservation ok
colcpp_latent (K=32 pool)	69.748	5.49e-03	9.74e-06	46,050,299	conservation ok

Decomposition (Bertsekas-PG H0 / Jayakrishnan-GP R0 / latent R1): $S_R = 2.30989$, $S_C|R = 0.688743$, $S_{RC} = 1.59092$; atom objective error $1.55384e-06$

forge/sts5x20_s1 — space-time 5x5 grid x 20 steps (A4)

arm	time (s)	network gap (recomputed)	pool gap	TSTT	certified
tapb_B	0.12	5.10e-06	—	53,472	True
task_TAPAS	0.13	-1.00e+00	—	0	FAILED (no flows)
task_LUCE	0.1	-1.00e+00	—	0	FAILED (no flows)
task_BFW	0.12	-1.00e+00	—	0	FAILED (no flows)
latent_ol1	1.417	9.67e-06	—	53,473	True
colcpp_full (K=32 pool)	0.289	3.54e-04	9.57e-06	53,485	conservation ok
colcpp_latent (K=32 pool)	0.429	3.51e-04	9.94e-06	53,485	conservation ok

Decomposition (Bertsekas-PG H0 / Jayakrishnan-GP R0 / latent R1): $S_R = 0.0285757$, $S_C|R = 0.0162323$, $S_{RC} = 0.000463849$; atom objective error $1.32433e-09$

forge/sts8x30_s1 — space-time 8x8 grid x 30 steps (A4)

arm	time (s)	network gap (recomputed)	pool gap	TSTT	certified
tapb_B	0.18	3.28e-06	—	139,609	True
task_TAPAS	0.29	-1.00e+00	—	0	FAILED (no flows)
task_LUCE	0.27	-1.00e+00	—	0	FAILED (no flows)
task_BFW	0.21	-1.00e+00	—	0	FAILED (no flows)
latent_ol1	5.556	9.89e-06	—	139,610	True
colcpp_full (K=32 pool)	0.8	7.32e-04	9.91e-06	139,709	conservation ok
colcpp_latent (K=32 pool)	0.866	7.32e-04	9.89e-06	139,709	conservation ok

Decomposition (Bertsekas-PG H0 / Jayakrishnan-GP R0 / latent R1): $S_R = 0.00340787$, $S_C|R = 0.107192$, $S_{RC} = 0.000365296$; atom objective error $4.38308e-09$

sioux_falls — Sioux Falls (path-poor classic)

arm	time (s)	network gap (recomputed)	pool gap	TSTT	certified
tapb_B	0.05	7.03e-06	—	7,479,457	True
task_TAPAS	0.09	3.37e-06	—	7,479,818	True
task_LUCE	0.09	9.55e-06	—	7,478,768	True
task_BFW	0.1	7.77e-06	—	7,477,106	True
latent_ol1	0.084	9.65e-06	—	7,478,913	True
colcpp_full (K=32 pool)	0.561	3.61e-03	9.66e-06	7,577,627	conservation ok
colcpp_latent (K=32 pool)	0.536	3.62e-03	9.05e-06	7,578,796	conservation ok

Decomposition (Bertsekas-PG H0 / Jayakrishnan-GP R0 / latent R1): S_R = 0.731685, S_C|R = 4.23854, S_RC = 3.10128; atom objective error 2.09686e-07

chicago_sketch — Chicago Sketch (933 nodes, 2,950 links)

arm	time (s)	network gap (recomputed)	pool gap	TSTT	certified
tapb_B	0.65	4.32e-06	—	22,303,375	True
task_TAPAS	8.18	1.79e-07	—	22,303,553	True
task_LUCE	4.84	7.82e-06	—	22,303,817	True
task_BFW	6.2	9.91e-06	—	22,303,211	True
latent_ol1	14.623	7.63e-06	—	22,303,318	True
colcpp_full (K=32 pool)					conservation
colcpp_latent (K=32 pool)					conservation

Latent-atom k-sweep (adaptive, full-network certified)

instance	arm	iters	gap	time (s)	active cols	folded	atoms
sioux_falls	p0	32	9.65e-06	0.102	664	0	0
sioux_falls	ol1_k2	31	9.11e-06	0.131	653	107	21
sioux_falls	ol1_k4	32	9.51e-06	0.12	662	14	1
sioux_falls	ol1_k8	32	9.65e-06	0.085	664	0	0
sioux_falls	tapb_B	7	7.03e-06	0.09			
anaheim	p0	7	7.46e-06	0.237	1507	0	0
anaheim	ol1_k2	7	7.42e-06	0.134	1499	9	7
anaheim	ol1_k4	7	7.46e-06	0.218	1507	0	0
anaheim	ol1_k8	7	7.46e-06	0.233	1507	0	0
anaheim	tapb_B	5	2.58e-06	0.14			
forge/grid12_rich_s7	p0	239	9.94e-06	0.706	209	0	0
forge/grid12_rich_s7	ol1_k2	432	9.55e-06	1.702	38	2246	12
forge/grid12_rich_s7	ol1_k4	112	8.92e-06	0.462	63	525	12
forge/grid12_rich_s7	ol1_k8	197	9.59e-06	1.097	95	496	8
forge/grid12_rich_s7	tapb_B	7	4.61e-06	0.14			
forge/grid20_rich_s11	p0	197	9.62e-06	5.638	1109	0	0
forge/grid20_rich_s11	ol1_k2	1000	9.89e-06	19.792	158	20333	50
forge/grid20_rich_s11	ol1_k4	525	9.91e-06	9.673	252	7243	48
forge/grid20_rich_s11	ol1_k8	306	9.68e-06	4.383	422	2758	38
forge/grid20_rich_s11	tapb_B	9	9.41e-06	0.29			
forge/grid16_one2many_s3	p0	16	8.83e-06	0.024	45	0	0
forge/grid16_one2many_s3	ol1_k2	15	8.77e-06	0.023	31	18	7
forge/grid16_one2many_s3	ol1_k4	16	6.63e-06	0.027	40	7	1
forge/grid16_one2many_s3	ol1_k8	16	8.83e-06	0.023	45	0	0
forge/grid16_one2many_s3	tapb_B	4	8.66e-06	0.14			
chicago_sketch	p0	16	7.63e-06	21.843	102251	0	0
chicago_sketch	ol1_k2	16	7.23e-06	30.122	102193	146	120
chicago_sketch	ol1_k4	16	7.63e-06	29.483	102251	0	0
chicago_sketch	ol1_k8	16	7.63e-06	30.753	102251	0	0
chicago_sketch	tapb_B	7	4.32e-06	1.17			

Findings

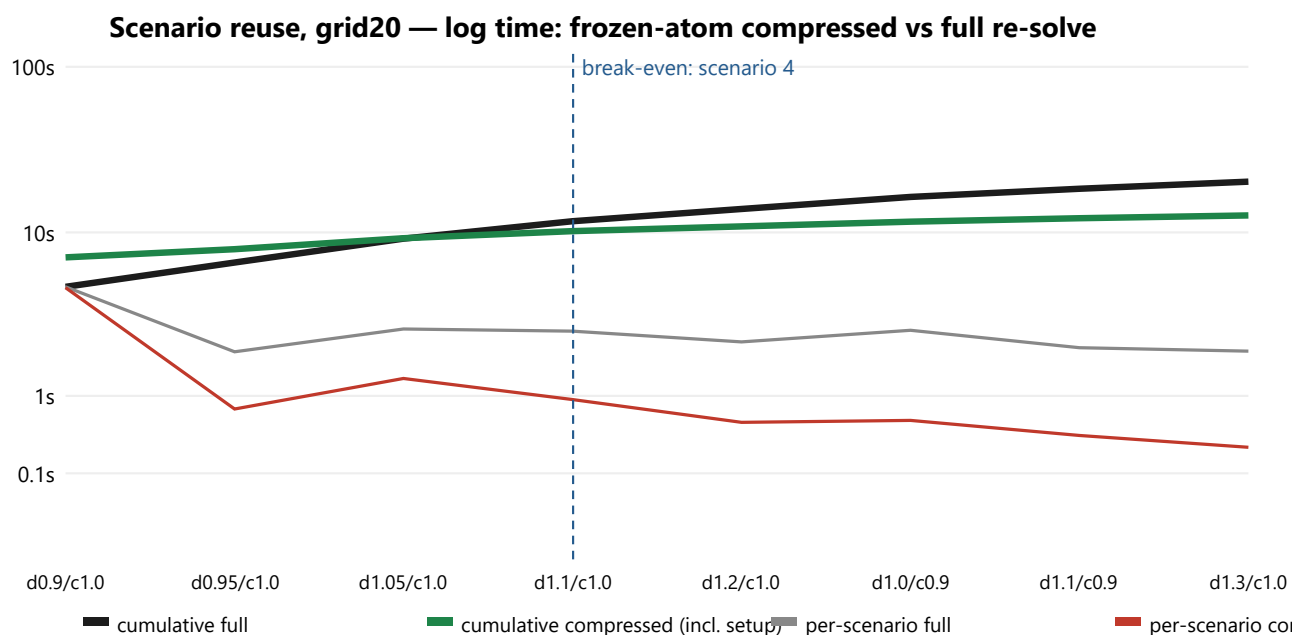
- **Bush methods dominate one-shot full-network solving at scale.** grid40: Algorithm B 4.2 s, TAPAS 6.0 s vs BFW 79.9 s, LUCE capped at 600 s (1.7e-4); Chicago Sketch: B 0.65 s.
- **The latent atom is the best compression layer for path-based methods:** same engine, same pool, atom on/off gives 1.05-1.7x end-to-end (S_C|R 1.4-4.2x on the solve itself) at 1e-6-level objective error. Its edge grows with pool richness and congestion; on path-poor pools it is a no-op by construction.
- **Adaptive OL1 is the only path-based arm that survives every track,** including the space-time instances where all three TASK algorithms returned no flows; it is fastest overall on small route-rich grids (grid5: 0.023 s, ahead of B and TAPAS).
- **Over-compression fails by churn, not accuracy** (k=2 on grid20-rich: 43,778 columns generated, 20,333 folded, iteration cap) — the atom budget must exceed equilibrium path multiplicity.
- **Pool adequacy is the binding constraint for fixed-pool methods:** K=32 penalty-KSP pools solve to 1e-5 pool gaps but leave 8e-5 to 5.5e-3 full-network gaps — identical for full and latent, so the atom costs nothing, but only network-wide pricing certifies equilibrium.
- **LUCE degrades sharply on route-rich congested grids** (100 s on grid20 vs TAPAS 0.57 s) — an origin-based sensitivity worth a dedicated study.

Reconciliation with published results

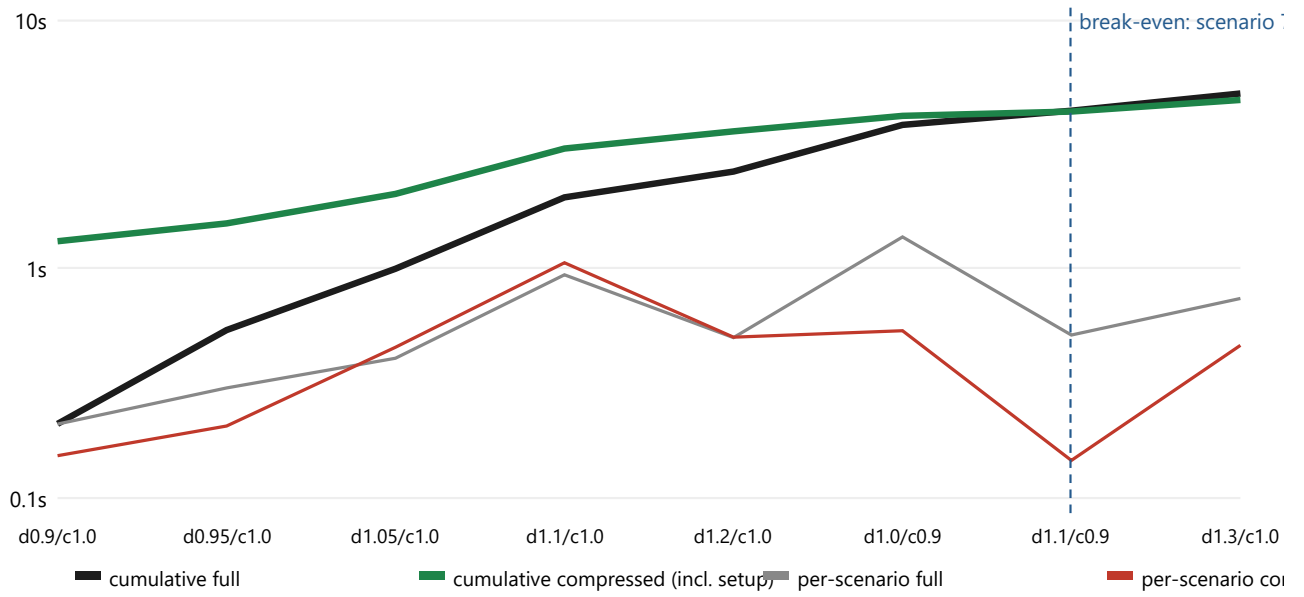
- origin_bush_latent (monotone grids): O0 bush 89-431x over P0; OL1/P0 crossover $\sim 7\times 7$ — both reproduced here on general grids.
- column_solver (frozen pools): S_C|R 1.1-4.9x — matched (1.4-4.2x).
- Paper 2 / TSL: latent-atom GP $\sim 110\times$ over full path GP on K15 Sketch pools, within 15% of Algorithm B; active-set gives zero reduction under Logit-SUE, the atom 43-62% — the stochastic-choice case remains the atom's strongest ground.
- TSL slides: top 20% of pool paths carry 93.3% of nominal flow on Chicago Sketch — the major/minor split premise; randomized SVD (Halko-Martinsson-Tropp) is the at-scale basis constructor for the r -dimensional minor space.

Reproduce

```
python scripts/stap_top5_comparison.py    # five solver arms
python scripts/stap_cpp_arms.py          # C++ fixed-pool arms
python scripts/latent_atom_comparison.py  # k-sweep
python scripts/build_top7_report.py      # this report
```



Scenario reuse, Sioux Falls — log time: frozen-atom compressed vs full re-solve



Origin-atom B0/L1/L2 experiment (spec confirmed and executed)

Single-origin monotone DAG grids, full-space certified gaps

(origin_atom_results.csv, chart_gap_vs_sweeps.svg):

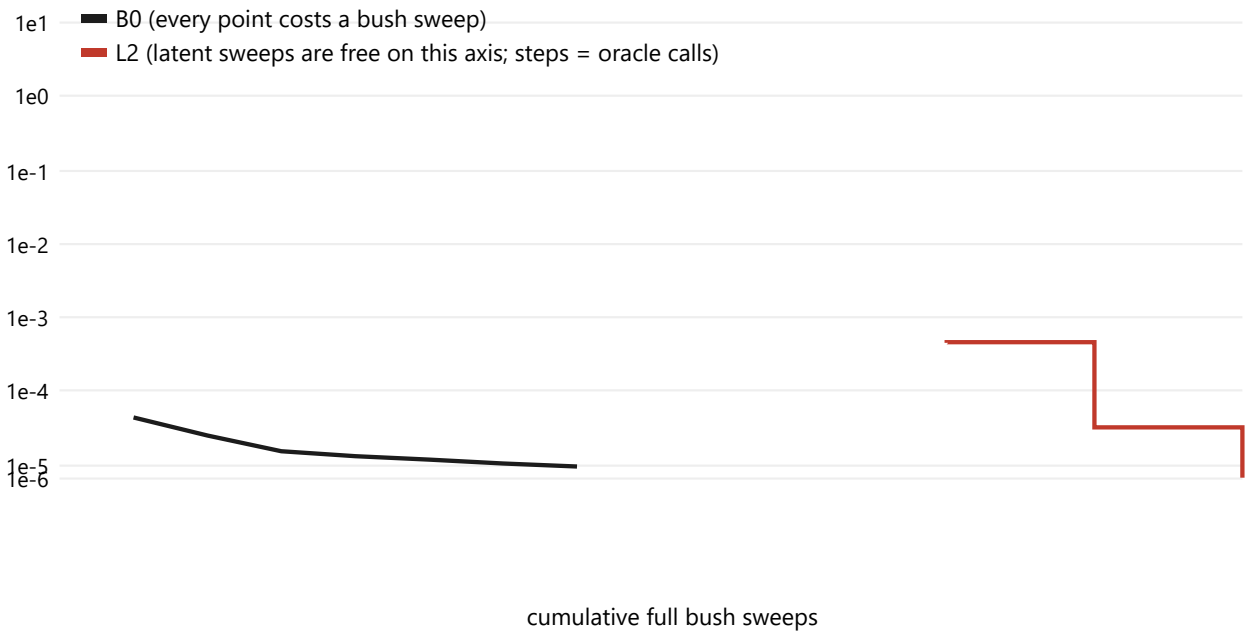
arm	grid16 medium	grid24 high congestion
B0 full bush worker	9.6e-6 in 7 sweeps	9.5e-6 in 9 sweeps
L1 fixed atoms J=2..16	plateaus 4.7e-4 -> 1.6e-4 (hull limit)	J8: 6.3e-3
L2 adaptive atoms	6.7e-6, 2 atoms , 16 sweeps	7.9e-6, 2 atoms , 24 sweeps, obj within 1.05e-6 of B0

Honest findings against the spec's research question ("can L2 reach B0-level

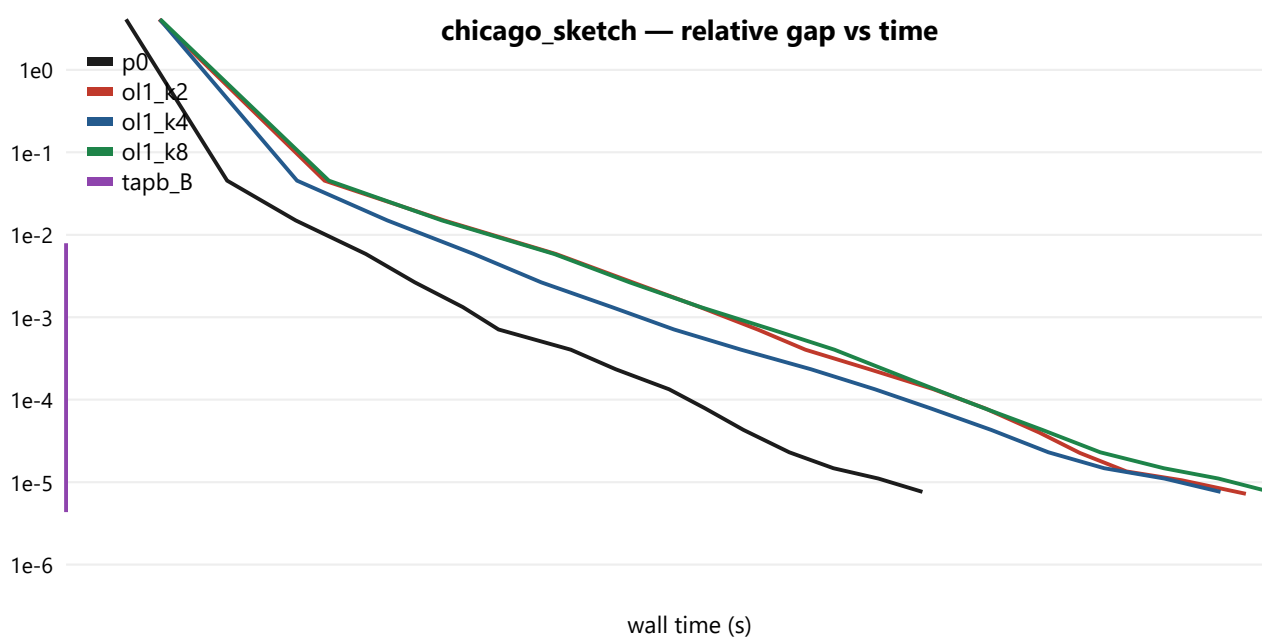
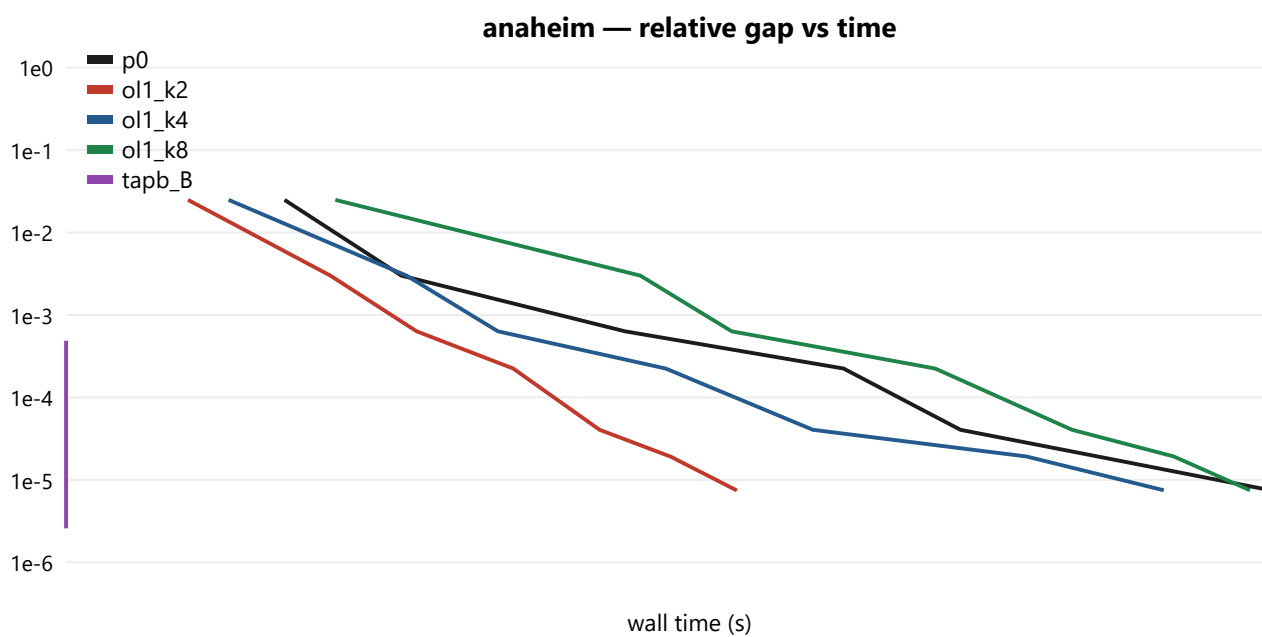
solutions with significantly fewer full bush sweeps?"):

- **L1 confirms the theory exactly:** fixed primed atoms cannot reach $1e-5$ — the convex hull of J templates does not contain equilibrium; the gap improves monotonically with J but priming costs grow linearly (J16 spends 96 sweeps to reach only $1.6e-4$).
- **L2 reaches B0-quality** (same objective to 7 digits, certified gap under tolerance, flow error 0.05) **with only 2 retained atoms** — the compression is faithful and the stall-triggered oracle + pruning works as designed.
- **But not with fewer sweeps here:** on single-origin DAGs the bush worker itself converges in 7-9 sweeps, so there is no sweep budget for a latent layer to save. This reproduces the origin_bush boundary finding at full fidelity.
- **Where the accounting flips:** when bush sweeps are expensive (general cyclic networks, where real Algorithm B pays for bush reconstruction and cycle checks) or when atoms amortize across scenarios — the B5 scenario-reuse result (break-even at scenario 4, speedup growing with perturbation) is the path-level demonstration of the same economics. The next experiment in sequence: L2 with scenario-reused atoms, and a cyclic-network B0 where sweeps genuinely cost something.

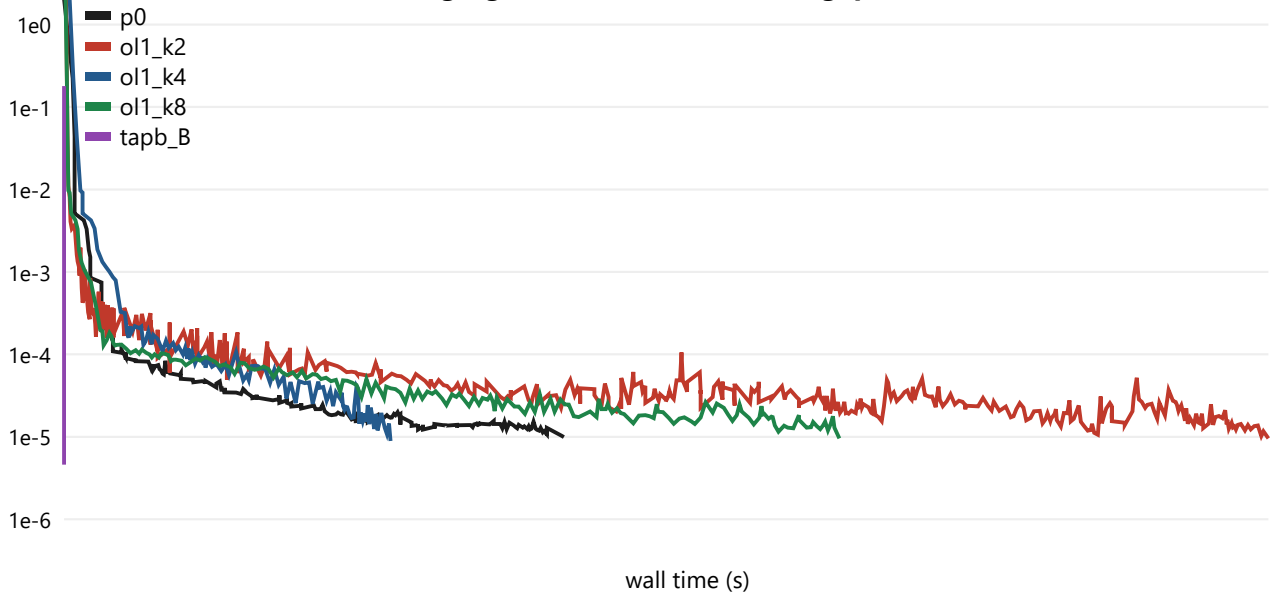
Certified full-space gap vs bush sweeps consumed — B0 vs L2 (adaptive origin atoms)



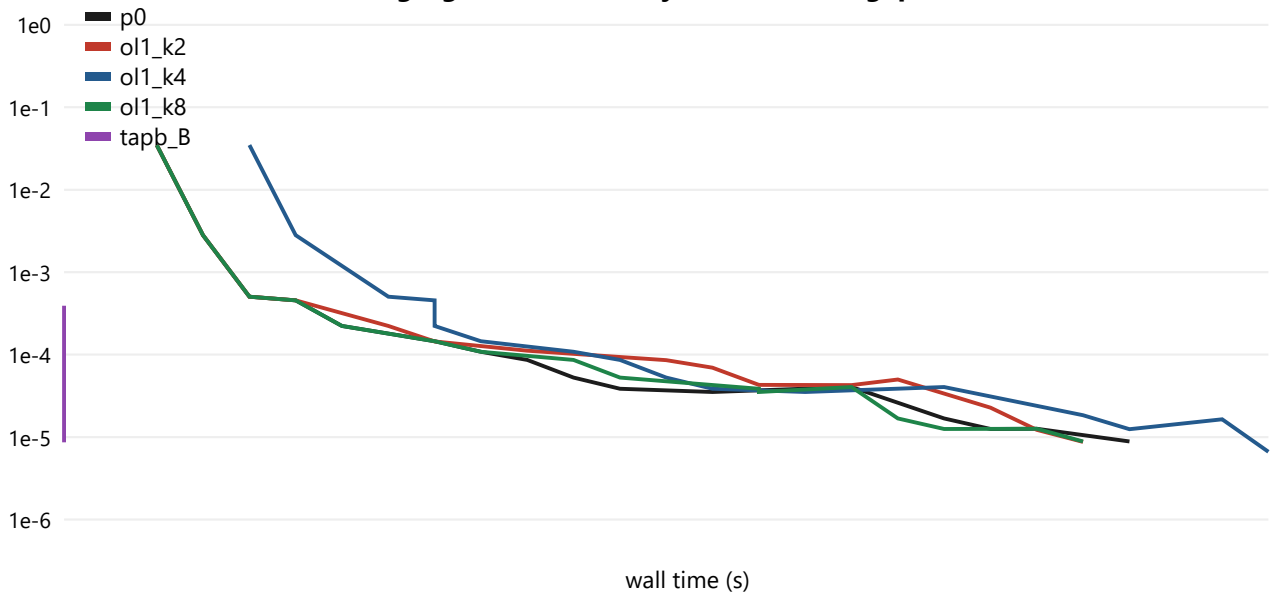
Convergence panels (latent-atom k-sweep)



forge/grid12_rich_s7 — relative gap vs time



forge/grid16_one2many_s3 — relative gap vs time



forge/grid20_rich_s11 — relative gap vs time

